

19. DDP & MLOps

Overview

Distributed Data Parallel (DDP) trains PyTorch models across processes with gradient synchronization. MLflow tracks experiments, parameters, and artifacts. We wire both together for reproducible multi-GPU (or single-GPU baseline) runs and model registry basics.

Lab: Lab 19.

Prior modules: Module 18.

Contents

1	Training at Scale	2
2	Data Parallelism	2
3	Launch and Process Group	2
4	Effective Batch Size	2
5	MLOps Lifecycle	2
6	Mixed Precision	2
7	Fault Tolerance	3
8	Lab	3
9	Gradient Accumulation	3
9.1	Model Serving SLOs	3
10	Extended Intuition	3
11	Numerical Walkthrough	4
12	PyTorch Implementation Notes	4
13	Local GPU Constraints	4
14	Debugging Checklist	4
14.1	Checkpoint Policy, LR Scaling, and MLflow Registry	5
15	Practice Problems	5

Module track

This module builds on **Module 18**. Read those PDFs first. References to other modules appear as **Module NN** (not page numbers, because each module is its own PDF).

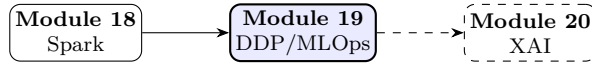


Figure 1: Module sequence (solid box: this PDF; dashed: typical next step).

1 Training at Scale

Large models (**Module 10**, **Module 04** (transformer block)) require multi-GPU training. Distributed Data Parallel (DDP) is the standard PyTorch pattern; MLOps operationalizes the lifecycle.

2 Data Parallelism

Replicate model on K GPUs; each processes minibatch $\mathcal{B}_k$:

$$\mathbf{g} = \frac{1}{K} \sum_{k=1}^K \nabla_{\theta} \mathcal{L}_k(\theta), \quad \theta \leftarrow \theta - \eta \mathbf{g}. \quad (1)$$

All-reduce synchronizes gradients each step.

Note. DDP overlaps communication with backward pass via gradient buckets, near-linear speedup to moderate K .

3 Launch and Process Group

`torchrun -nproc_per_node=K train.py spawns ranks 0, ..., K-1.` Each rank: `dist.init_process_group(backend="nccl")`, wrap model in `DistributedDataParallel`.

4 Effective Batch Size

Global batch $B_{\text{global}} = B_{\text{local}} \times K$. Linear scaling rule: $\text{lr} \propto B_{\text{global}}$ when batch grows (with warmup).

5 MLOps Lifecycle

1. Experiment tracking (MLflow, W&B): log metrics, artifacts
2. Model registry: versioned staging $\rightarrow$ production
3. CI/CD: data validation, unit tests, integration tests
4. Monitoring: drift detection, latency, error rates

6 Mixed Precision

Automatic Mixed Precision (AMP) uses FP16 forward/backward with FP32 master weights:

$$\mathcal{L}_{\text{scaled}} = s \cdot \mathcal{L}, \quad \text{gradients unscaled before } \theta \leftarrow \theta - \eta \nabla \theta. \quad (2)$$

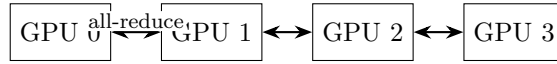
Reduces memory $\approx 2\times$, increases throughput on Tensor Cores.

Table 1: MLOps tooling layers.

Layer	Examples
Orchestration	Airflow, Prefect, Kubeflow
Tracking	MLflow, Weights & Biases
Serving	TorchServe, Triton, BentoML
Infra	Docker, K8s, Terraform

7 Fault Tolerance

Checkpoint `state_dict` every N epochs; elastic training restarts failed ranks. Contrasts with Spark fault tolerance in **Module 18**.



Synchronized gradient average each step

Figure 2: DDP all-reduce across GPUs.

8 Lab

Lab 19 launches multi-GPU fine-tune with DDP; log experiments to MLflow per table 1. Deploy with monitoring hooks for XAI in **Module 20**.

9 Gradient Accumulation

Simulate large batch when memory-limited:

$$\mathbf{g} = \frac{1}{A} \sum_{a=1}^A \nabla_{\theta} \mathcal{L}_a, \quad \text{update every } A \text{ micro-batches.} \quad (3)$$

Combines with DDP: global batch = $B_{\text{local}} \times K \times A$.

9.1 Model Serving SLOs

Track p50/p99 latency, throughput, and GPU utilization; autoscale replicas on queue depth.

10 Extended Intuition

Distributed Data Parallel (DDP) replicates the model on each GPU/process, runs independent forward-backward passes, and all-reduces gradients so every replica stays synchronized. Effective batch size scales with world size: $2 \text{ GPUs} \times \text{batch } 8 \text{ per GPU} = \text{effective batch } 16$ for BatchNorm statistics (note: BN behaves differently across GPUs unless converted to SyncBatchNorm).

MLflow tracks experiments: parameters, metrics per step, artifacts (checkpoints, plots), and model registry stages (Staging, Production). Reproducibility requires logging git commit, seed, library versions, and data snapshot ids alongside learning rate schedules.

Even on a single GPU, structuring training like DDP (config files, checkpointing, MLflow) pays off before scaling out. Spark preprocessing from **Module 18** can feed parquet shards consumed by a PyTorch **Dataset** in multi-worker DataLoaders.

Fine-tuning workflows from **Module 11** combine naturally with DDP when moving LoRA training to a multi-GPU machine.

11 Numerical Walkthrough

Single-process baseline: BERT-style classifier, $B = 8$, $T = 128$, 1000 steps, $\text{lr} = 2 \times 10^{-5}$, 1 GPU (1650).

DDP on 2 GPUs (lab simulation may use 1 GPU + CPU note): per-GPU batch 8, gradient all-reduce averages grads across processes. Wall time per step ideally $\approx 0.55 \times$ single-GPU step (not perfect 0.5 due to comm overhead).

MLflow log example: `params {lr: 2e-5, batch: 8, world: 2}`, metrics `train_loss` every 10 steps, artifact `checkpoint-500.pt`.

`torchrun -nproc_per_node=2 train.py` sets `LOCAL_RANK` environment variables consumed by `dist.init_process_group(backend="nccl")`. On Windows, gloo backend may substitute for nccl; Linux/WSL preferred for CUDA DDP labs.

Checkpoint size 440 MB fp32 BERT-base; log as MLflow artifact with `mlflow.log_artifact(path)`.

12 PyTorch Implementation Notes

- ```
dist.init_process_group("nccl"); local_rank = int(os.environ["LOCAL_RANK"]); torch.cuda.set_device(local_rank)
```
- .
- Wrap model: `model = DDP(model, device_ids=[local_rank]);` forward unchanged.
- `DistributedSampler(dataset)` and `sampler.set_epoch(epoch)` each epoch for shuffling.
- Only rank 0 logs/saves: `if dist.get_rank()==0: mlflow.log_metric(...)`.
- `mlflow.start_run(); mlflow.log_params(vars(args)); mlflow.pytorch.log_model` for registry.

```
model = DDP(model.cuda(local_rank), device_ids=[local_rank])
sampler = DistributedSampler(train_set, shuffle=True)
loader = DataLoader(train_set, batch_size=8, sampler=sampler)
```

## 13 Local GPU Constraints

True multi-GPU DDP needs multiple devices; on one 1650, run DDP code path with `world_size=1` or CPU gloo for structure validation. 4 GB per GPU: keep per-replica batch small; global batch = local batch  $\times$  world size.

MLflow local tracking store (`./mlruns`) is lightweight; artifact uploads from large checkpoints can fill disk. Use `save_top_k` checkpoint policy; fp16 checkpoints halve artifact size.

## 14 Debugging Checklist

- Hang at init: firewall blocking gloo/tcp; mismatched `world_size` in launch vs script.

- Duplicate data per GPU: forgot `DistributedSampler` or `shuffle=False` on `DataLoader` incorrectly.
- Loss diverges vs single-GPU: effective lr should scale with global batch if you linearly scale lr rule.
- MLflow empty metrics: logging from non-zero rank only by mistake.
- NCCL error on Windows: switch to WSL2 or use gloo with CPU for code debugging only.

### 14.1 Checkpoint Policy, LR Scaling, and MLflow Registry

Gradient all-reduce in DDP averages gradients across ranks after backward; every replica applies the same optimizer step, keeping weights identical (up to floating-point drift). Linear LR scaling rule: if you double global batch size, double learning rate up to a warmup cap; misf scaling is a common reason multi-GPU loss diverges while single-GPU trains fine. Warmup for first 500 to 1000 steps stabilizes large effective batches before full lr kicks in. `SyncBatchNorm` converts per-GPU batch statistics into global stats; necessary when per-GPU batch is small ( $B \leq 4$ ) and BatchNorm layers dominate.

Checkpoint strategy: save every  $N$  epochs, keep best-by-val-metric and latest; name files with epoch, step, and metric (`epoch3-step1200-val0.91.pt`). Only rank 0 should write checkpoints to avoid file corruption from concurrent writers. MLflow experiment naming convention (`project/task/date/runid`) saves hours when comparing 20 hyperparameter sweeps later. Log artifacts beyond weights: confusion matrices, sample predictions, `requirements.txt`, and git SHA via `mlflow.log_param("git_sha", sha)`.

Model Registry stages (`Staging`, `Production`) gate promotion behind val metrics and manual review. Register with `mlflow.register_model(uri, name)` after a successful run; load in serving with `models:/name/Production`. For LoRA fine-tunes from **Module 11**, log both adapter files and base model id so reproducibility does not depend on Hugging Face cache state alone. DDP adds communication overhead; at 2 GPUs expect  $1.6$  to  $1.9\times$  throughput, not  $2\times$ ; profile before scaling cluster size. Combine Spark featurization (**Module 18**) with DDP training by writing parquet shards per rank slice or using `DistributedSampler` on exported indices.

Torchrun launch template for local debugging:

```
torchrun --nproc_per_node=2 train.py --epochs 3
if "LOCAL_RANK" in os.environ:
 dist.init_process_group("nccl")
 local_rank = int(os.environ["LOCAL_RANK"])
 torch.cuda.set_device(local_rank)
```

Use `find_unused_parameters=False` in DDP when every parameter receives a gradient; setting True adds comm overhead for conditional branches. Environment variables `MASTER_ADDR` and `MASTER_PORT` must match across processes; default localhost works for single-node multi-GPU. MLflow tags (`mlflow.set_tag("stage", "baseline")`) filter runs faster than parsing param dicts in the UI when comparing 50+ experiments. Resume training: load checkpoint on all ranks before wrapping in DDP; broadcast state dict from rank 0 if only rank 0 has the file on shared filesystem. Health check endpoint in serving should read model version from MLflow registry metadata, not hardcoded path strings in Flask config. Gradient accumulation with DDP: divide loss by `accumulation_steps` before `backward()`, step optimizer only every  $k$  micro-batches; effective batch scales without extra GPU memory.

## 15 Practice Problems

**Problem 1.** Effective batch size with 4 GPUs, batch 6 each, gradient accumulation 2 steps.

**Problem 2.** Launch training with `torchrun`; verify each rank sees disjoint train shards.

**Problem 3.** Log learning rate and val accuracy to MLflow; compare two runs in UI.

**Problem 4.** Register best checkpoint in MLflow Model Registry with alias `staging`.

Wire up in **Lab 19**; follows Spark ETL in **Module 18**; deploy with monitoring in **Module 20**.

## Source certification

This module lists where each core definition, equation, figure, and summary table comes from. Pedagogical simplifications (lab batch sizes, epoch counts, illustrative plots) are labeled in extension sections and are not claimed as optimal production settings.

**Table 3:** Certified topic map for Module 19.

| Topic                      | Primary source | Where in this PDF |
|----------------------------|----------------|-------------------|
| PyTorch DDP pattern        | [3]            | Section 2         |
| MLflow experiment tracking | [4]            | Section 5         |

Full bibliographic entries appear below. Figures are schematic unless a caption cites a specific paper figure. If a statement is not listed here, treat it as general ML practice or lab-specific guidance, not a cited theorem.

## References

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [2] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *ICLR*, 2015.
- [3] Shen Li, Yanli Zhao, Rohan Varma, et al. Pytorch distributed: Experiences on accelerating data parallel training. 2020.
- [4] Matei Zaharia, Aaron Chen, Andrew Davidson, et al. Accelerating the machine learning lifecycle with MLflow. 2018.